前置知識與目標

- 前置知識為集合、樹、堆疊、佇列、優先權佇列與歸納證明。
- 能選擇圖表示，證明搜尋與貪婪選邊的關鍵不變量，依邊權條件選擇最短路徑算法。
- 能區分最小生成樹、最短路徑、拓樸次序及關鍵路徑的不同最佳化目標。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Graph ADT 與問題契約

- 圖 $G=(V,E)$。有向邊 (u,v) 有方向，無向邊 $\{u,v\}$ 不區分方向。權重為映射 $w:E\to R$。
- 本章除另註外採無自環、無平行邊的圖。演算法可延伸到 multigraph，但表示與去重政策要明示。
- ADT 可提供頂點列舉、鄰居列舉、加邊與權重查詢。不同表示對這些操作有不同成本。
- 邊權 0 是合法數值。不可在加權矩陣中同時用 0 表示「無邊」。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

鄰接矩陣與鄰接串列

表示	空間	判定特定邊	列舉 u 的鄰居
矩陣	$\Theta(V^2)$	$O(1)$	$\Theta(V)$
未排序串列	$\Theta(V+E)$	$O(\deg(u))$	$\Theta(\deg(u))$

此表以 `V=|V|`, `E=|E|` 簡寫。無向圖每邊通常儲存兩筆鄰接項。串列中加邊是否需檢查重複，會改變插入成本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

BFS 的層數不變量

1. 起點 s 設 `dist[s]=0` 並入列，其餘為未發現。
2. 出列 u，對每個未發現鄰點 v 設 `dist[v]=dist[u]+1`、`parent[v]=u` 並入列。
3. 在入列時標記發現，避免同一點多次入列。

佇列的待處理距離保持非遞減。首次發現 v 時，任何更短路徑的前驅都應更早被處理，故該距離為最少邊數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

DFS、時間戳與有向環

- DFS 把頂點分為白色未發現、灰色正在處理、黑色已完成。遞迴堆疊對應灰色路徑。
- 有向圖中，指向灰色祖先的 back edge 證明存在有向環。指向黑色頂點本身不表示有環。
- 完成時間可支援拓樸排序。若圖無環，依完成時間遞減排列是一個合法拓樸次序。
- 只從單一起點搜尋不保證涵蓋全圖。要建立搜尋森林，需對剩餘白色頂點再次啟動。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

搜尋成本與可重現追蹤

- 鄰接串列中，每個頂點與每筆鄰接項處理固定次數，完整 BFS/DFS 為 $\Theta(V+E)$，輔助空間 $O(V)$。
- 矩陣版本對每點掃整列，完整搜尋為 $\Theta(V^2)$。不可不加條件套用串列成本。
- 原創無向圖邊為 `AB,AC,BD,CD,DE`，鄰居按字母順序。從 A 做 BFS 為 `A,B,C,D,E`。
- 同圖遞迴 DFS 為 `A,B,D,C,E`。走訪順序受鄰居次序影響，距離與可達性結論不受影響。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

BFS 與 DFS 的完整狀態機

```
BFS(s):
    for v in V: dist[v]=infinity; parent[v]=none
    dist[s]=0; Q.push(s)
    while Q not empty:
        u=Q.pop_front()
        for v in adj[u]:
            if dist[v]==infinity:
                dist[v]=dist[u]+1; parent[v]=u; Q.push(v)
DFS(u):
    color[u]=gray; discover[u]=++clock
    for v in adj[u]:
        if color[v]==white: parent[v]=u; DFS(v)
        else if directed and color[v]==gray: report back edge
    color[u]=black; finish[u]=++clock
```

對無向AB,AC,BD,CD,DE、字母序鄰居，BFS出列後的queue為：A後[B,C]；B後[C,D]；C後[D]；D後[E]；E後[]。d=(0,1,1,2,3)，parent=(無,A,A,B,D)。DFS時間戳為A:(1,10)、B:(2,9)、D:(3,8)、C:(4,5)、E:(6,7)，parent C=D、E=D。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

BFS 與 DFS 的完整狀態機 （續）

有向圖灰邊表示回祖先形成環；無向圖必須忽略回parent的同一條樹邊，不能把每條無向邊的反向鄰接項當成新環。完整森林先將全圖白化，再對每個白點呼叫DFS；單源BFS不會替其他分量定距離。深度、queue與parent皆需Θ(V)最壞空間，不包含輸入。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

MST 的目標與割性質

- 對連通無向加權圖，生成樹含全部頂點且恰有 V−1 邊；MST 最小化總權重，允許負權。非連通圖改求各分量的最小生成森林。
- 維持不變量「存在一棵 MST T，包含目前全部已選邊 F」。割 (S,V\S) 尊重 F，意指 F 沒有跨割邊。令 e 是一條跨割最輕邊。

若 e∈T，不變量直接延續。否則 T+e 有唯一環；該環除 e 外還有跨同一割的邊 f。因割尊重 F，f∉F；又 w(e)≤w(f)。故 T′=T−f+e 仍是生成樹，權重不大於最優 T，只能同為最優，並包含 F∪{e}。交換沒有丟掉先前選擇，這才證明 e 對目前 F 安全。

不能只證「每條邊各自屬於某棵 MST」。三角形三邊權皆 1，每條各自出現在某棵 MST，但三條一起成環，沒有任何 MST 能同時包含它們。安全性必須相對整組 F 而言。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Prim 逐步擴張一個頂點集合

- 令 S 為已納入頂點，為每個外部點保留連到 S 的最小邊權 `key[v]` 與對應 parent。
- 每次選最小 key 的外部點加入 S，再更新鄰邊。選入邊是割 `(S,V\S)` 的最輕邊。
- 二元堆與鄰接串列為 `O((V+E)log V)`；連通非單點圖可簡寫 `O(E log V)`。矩陣線性挑點版本為 $O(V^2)$。
- key 是「連到目前樹的單邊成本」，不是從起點累計距離，因此不可與 Dijkstra 的 dist 混用。

歸納從 F=∅ 開始。Prim 的 F 全在目前 S 內，故割 (S,V\S) 尊重 F；取其最輕跨邊可由安全邊定理維持「某棵 MST 包含 F」。連通圖最後有 V−1 條無環邊，F 本身就是那棵 MST。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Kruskal 與互斥集合

1. 將全部邊依權重排序。
2. 從輕到重處理邊 `(u,v)`。若兩端在不同分量，加入邊並 union。
3. 連通圖選滿 `V-1` 邊即可停止。否則完成最小生成森林。

使用 path compression 及 union by rank 的互斥集合，m 次操作可用 `O(m α(V))` 攤銷界。排序通常主導 `O(E log E)`。此處 α 為反 Ackermann 函數，與雜湊負載率不是同一符號。

Kruskal 令 S 為目前 F 中含 u 的整個連通分量；待選 e=(u,v) 兩端不同分量，故 e 跨此割且割尊重 F。若有更輕跨邊，它已較早被處理，當時兩端也不可能同分量，應已加入並合併，與現在仍跨割矛盾。因此 e 是該割最輕邊，安全邊定理維持同一包含不變量。Θ(V) 初始化不能省略；一般總界為 O(V+E log(E+1)+E α(V))，DSU 另占 Θ(V) 空間。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

互斥集合的完整森林操作

互斥集合（disjoint-set union, DSU）表示頂點的分割。每個非空集合恰有一個根，以 parent[root]=root 表示；find 回傳代表根，union 只合併兩個原本不同的集合。陣列在 make_set 前配置 V 格，頂點 ID 必須有效。

```
make_set(v): parent[v]=v; rank[v]=0
find(v):
    r=v
    while parent[r]!=r: r=parent[r]
    while parent[v]!=v:
        next=parent[v]; parent[v]=r; v=next
    return r
union(u,v):
    a=find(u); b=find(v)
    if a==b: return false
    if rank[a]<rank[b]: swap(a,b)
    parent[b]=a
    if rank[a]==rank[b]: rank[a]+=1
    return true
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

互斥集合的完整森林操作 （續）

第二個 find 迴圈必先保存 next，否則覆寫 parent 後會丟掉尚未壓縮的路徑。rank 是合併歷史的高度上界，不是壓縮後的實際高度；壓縮後不重新降低 rank。union 的回傳值恰好可作為 Kruskal 是否收邊的條件。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

互斥集合的逐輪狀態與可自足成本界

沿同一 MST 的選邊次序 BC、AB、DE、CD 呼叫 union；rank 相等時保留第一個根。下表 parent 欄均依 A,B,C,D,E 排列。

已完成操作	parent[A..E]	根及其 rank
五次 make_set	[A,B,C,D,E]	A0,B0,C0,D0,E0
union(B,C)	[A,B,B,D,E]	A0,B1,D0,E0
union(A,B)	[B,B,B,D,E]	B1,D0,E0
union(D,E)	[B,B,B,D,D]	B1,D1
union(C,D)	[B,B,B,B,D]	B2
find(E)	[B,B,B,B,B]	B2

最後一次 find 的第一輪走 E→D→B，第二輪把 E 直接接 B；沒有把 E 的集合改掉。再遇 BD、AC、CE 時兩端都回 B，union 回 false，不能增加 rank，也不能再收邊。

互斥集合的逐輪狀態與可自足成本界 （續）

正確性由森林不變量歸納：find 只接往同一棵樹的祖先根，所以不改分割且不產生環；union 只把一個根接到另一個根，恰合併兩集合。每個非根到父的 rank 嚴格增加：低 rank 接高 rank，平手時勝根 rank 加一；壓縮接更高祖先仍保持。初始 rank0 集合恰含一點。rank 為 r 的根所代表集合至少含 2^r 點，因升 rank 只發生在兩個至少各 2^{r-1} 點的集合合併。因此 $\mathrm{rank} \le \lfloor \log_2 V \rfloor$，路徑長與單次 find/union 皆為 O(1+log(V+1))，初始化 Θ(V)、空間 Θ(V)。這已完整支持 Kruskal 的 O(V+E log(E+1)+E log(V+1)) 上界。

若進一步使用反 Ackermann 分層定理，V 次 make_set 後的 m 次 find/union 有更細的 O(V+m α(V)) 序列界；該精細定理不由上述 rank 高度證明推出。本章的演算法與較鬆對數界不依賴省略的精細證明，排序與所有頂點初始化仍需計入。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

MST 追蹤與反例

- 邊權為 AB=2,AC=5,BC=1,BD=4,CD=3,CE=7,DE=2。
- Kruskal 可依序選 BC,AB,DE,CD，總權重 1+2+2+3=8。平手順序可不同，合法最優總權重相同。
- Prim 從 A 可選 AB,BC,CD,DE，得到同一組邊但順序不同。
- MST 不保證起點到各點最短。例如三角形 AB=2,BC=2,AC=3 的 MST 使 A 到 C 為 4，原圖最短距離為 3。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

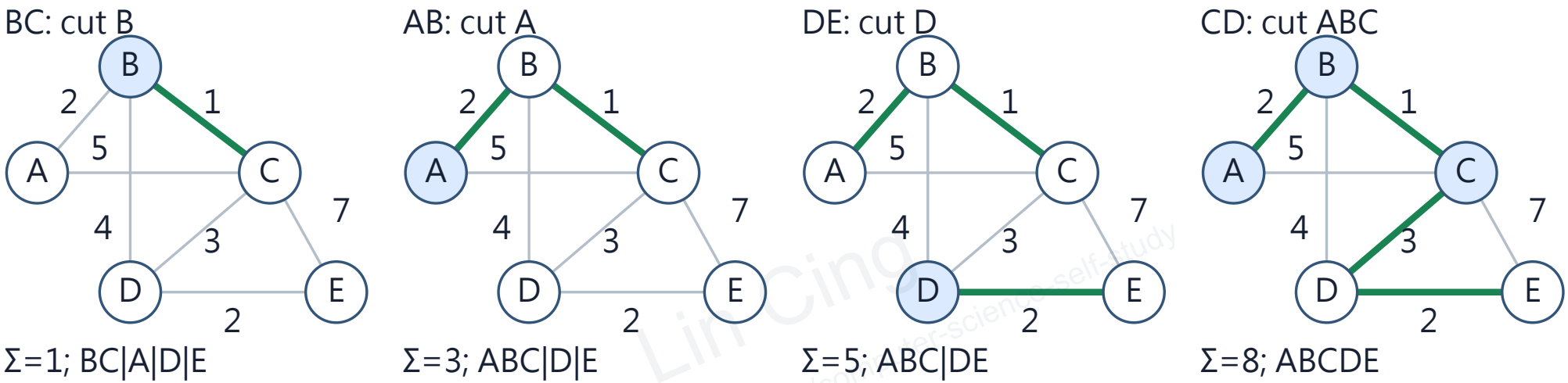
Prim 與 Kruskal 的完整選擇與狀態

```
Prim(s):
    key[*]=infinity; parent[*]=none; key[s]=0
    while some vertex is unselected:
        u=unselected vertex with minimum key
        if key[u]==infinity: start a new component or report disconnected
        select u
        for (u,v,w) in adj[u]:
            if v unselected and w<key[v]: key[v]=w; parent[v]=u
Kruskal():
    make_set(v) for all v; result=[]
    for (u,v,w) in sorted_edges:
        if find(u)!=find(v): result.append((u,v,w)); union(u,v)
    return result
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Prim 與 Kruskal 的完整選擇與狀態 (續)

Kruskal：藍為割側，綠為累積選邊



同一AB2,AC5,BC1,BD4,CD3,CE7,DE2圖，Prim從A的外部key依序：選A後B2,C5,D∞,E∞；選B後C1,D4,E∞；選C後D3,E7；選D後E2；選E結束。parent依次B←A、C←B、D←C、E←D，邊總重8。這些key從5降1、7降2，代表單邊連結成本，不是累加距離。

Prim 與 Kruskal 的完整選擇與狀態 (續)

Kruskal完整排序BC1,AB2,DE2,CD3,BD4,AC5,CE7；前四條各合併不同分量，後三條兩端已同分量而拒絕。DSU只負責環與分量，不負責權重最優；最優性來自前述割交換定理。相等權重可有不同MST，不能要求唯一樹。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

最短路徑與鬆弛

- 路徑長度為邊權和，δ(s,v) 是 s 到 v 的最小路徑長度。不可達記為 +∞。
- 若 s 可達某個負環，且負環又可達 v，則 v 的路徑長度可無界下降，沒有有限最短值。
- 鬆弛 (u,v) 為若 d[u] 有限且 d[u]+w(u,v)<d[v]，就更新 d[v] 與 parent。
- 每個有限 d 都是某條已發現路徑長度，因此是最短距離的上界。機器整數相加需另防溢位。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Dijkstra 的非負權條件

- 初始化 d[s]=0。每次選出尚未確定且 d 最小的 u，將 u 標為 settled，再鬆弛其出邊。
- 正確性關鍵是尚未處理路徑的後續邊不會使長度下降。這要求所有相關邊權非負。
- 反例 `s→a=2, s→b=5, b→a=-4`。若先把 a 的距離 2 永久確定，就錯過真正距離 1。
- 零權邊合法。不可把「正權」誤寫成必要條件。負權圖應改用符合條件的演算法。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Dijkstra 原創 trace 與實作差異

有向邊 `s→a=4,s→b=1,b→a=2,a→t=1,b→t=6`。

settled	d[a]	d[b]	d[t]
s	4	1	∞
b	3	1	7
a	3	1	4
t	3	1	4

indexed binary heap 為 `O((V+E)log V)`。採重複入堆而不 decrease-key 時，要跳過舊距離，空間可達 `O(V+E)`，時間保守寫 `O((V+E)log(V+E))`。

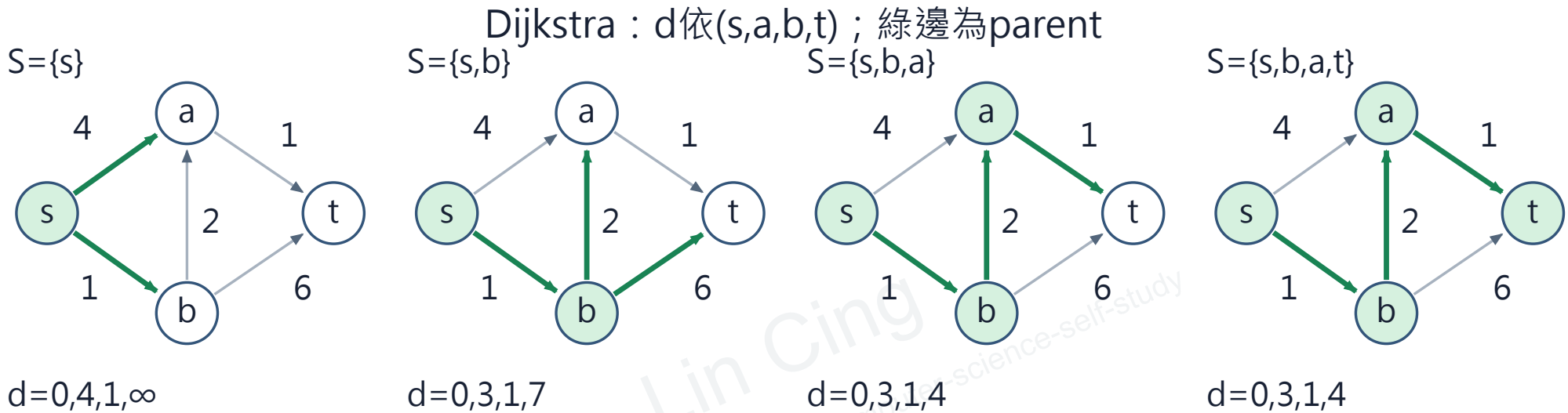
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Dijkstra 的優先佇列、舊項與前驅

```
Dijkstra(s):
    require all relevant edge weights>=0
    d[*]=infinity; parent[*]=none; d[s]=0
    H.push((0,s)); settled={}
    while H not empty:
        (du,u)=H.pop_min()
        if du!=d[u] or u in settled: continue
        settled.add(u)
        for (u,v,w) in adj[u]:
            if d[u]+w<d[v]:
                d[v]=d[u]+w; parent[v]=u; H.push((d[v],v))
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Dijkstra 的優先佇列、舊項與前驅 （續）



同一s→a4,s→b1,b→a2,a→t1,b→t6圖，初H=[(0,s)]。取s後H含(1,b),(4,a)；取b後加入(3,a),(7,t)；取a=3後加入(4,t)。舊(4,a)再彈出時因du≠d[a]而跳過；t=4確定後舊(7,t)也跳過。最終d=(s0,a3,b1,t4)，parent a=b,b=s,t=a。

Dijkstra 的優先佇列、舊項與前驅 （續）

證明settle u正確：取一條更短s→u路徑上第一個未確定點y，其前驅x已確定且已鬆弛，故d[y]≤該路徑前綴長；非負後續邊使該前綴≤整條更短路長，與u為最小d矛盾。負邊反例s→a2,s→b5,b→a−4沒有負環卻破壞這一步。距離相等時不必任意換parent，避免零權邊下人為形成前驅循環。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Bellman–Ford 的邊數歸納

- 做 V-1 輪全邊鬆弛。以同步陣列版本而言，第 k 輪後得到至多 k 邊的最短 walk 長度。
- 以原陣列立即更新的版本可能提早傳播，但每輪後至少已涵蓋所有至多 k 邊路徑，所以同樣正確。
- 沒有可達負環時，最短路徑可取簡單路徑，至多 V-1 邊。再一輪仍可鬆弛，即證明 s 可達負環。
- 原地鬆弛版時間 O(V+VE)；本章同步版每輪複製 V 格，時間 O(V(V+E))，另用 O(V) 距離與 parent 空間。只從 s 初始化，偵測的是 s 可達負環，不能宣稱找出全圖所有負環。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Floyd–Warshall 的中介點遞迴

- 在沒有相關負環的條件下，定義 `D^(k)[i,j]` 為中介點只允許 `{0,…,k-1}` 的最短距離；若允許可重複經過負環的 walk，有限最小值本身就不存在。
- 遞迴式為 `D^(k+1)[i,j]=min(D^(k)[i,j],D^(k)[i,k]+D^(k)[k,j])`。
- 初始化對角為 0，直接邊用其權重，其餘為無窮大。迴圈順序以 k 在最外層，才能對應中介點不變量。
- 時間 `Θ(V³)`、空間 `Θ(V²)`。結束後 `D[v,v]<0` 表示有負環；若 i 可達這樣的 v 且 v 可達 j，該 i→j 的 walk 長度無下界，不能把陣列中的有限值當成有限最短距離。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Bellman–Ford 邊數層與 Floyd 中介點層

```
BellmanFord_synchronous(s):
    d[*]=infinity; d[s]=0
    repeat V-1 times:
        next=copy(d)
        for (u,v,w) in E:
            if finite(d[u]): next[v]=min(next[v],d[u]+w)
        d=next
    if any finite(d[u]) and d[u]+w<d[v]: report reachable_negative_cycle
Floyd():
    D=direct_edges_and_zero_diagonal
    for k in V:
        for i in V:
            for j in V:
                if finite(D[i,k]) and finite(D[k,j]):
                    D[i,j]=min(D[i,j],D[i,k]+D[k,j])
```

負邊無環例s→a2,s→b5,b→a−4,a→t3的同步BF距離(s,a,b,t)：第0輪(0,∞,∞,∞)，第1輪(0,2,5,∞)，第2輪(0,1,5,5)，第3輪(0,1,5,4)，額外輪無改善。原地版本可能第1輪已達最終值，不能把同步等號不變量誤套到原地更新。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Bellman–Ford 邊數層與 Floyd 中介點層 （續）

Floyd對非負例以(s,a,b,t)排序，初矩陣為 `[[0,4,1,∞],[∞,0,∞,1],[∞,2,0,6],[∞,∞,∞,0]]`。k=s無變；k=a使D[s,t]=5、D[b,t]=3；k=b使D[s,a]=3、D[s,t]=4；k=t無變。終矩陣 `[[0,3,1,4],[∞,0,∞,1],[∞,2,0,3],[∞,∞,∞,0]]`，其source=s列與Dijkstra一致。

回到本節的負邊例（b→a權重−4），再加入a→b權重1後，a↔b環權−3；不是在剛才b→a權重2的非負例上加邊。BF額外輪找到可再改善邊後，從其受影響端點向外搜尋，才能把a,b,t標為−∞，s仍0。Floyd若D[k,k]<0，僅對可達i→k且k→j的pair標−∞。布林「有負環」不等於完整分類，distance陣列有限數不能直接當答案。

同步版的 copy(d) 不能藏在邊鬆弛成本內。對 $V \geq 2$ 且 $E=0$ 的合法圖，固定執行 $V-1$ 輪，單是複製便寫入 $V(V-1)$ 格，為 $\Theta(V^2)$；原地版只需 $\Theta(V)$ 初始化，空邊迴圈共 $O(V)$。一般同步版每輪 $\Theta(V+E)$，故 $\Theta(V(V+E))$；原地版為 $O(V+VE)$。只有另有 $E=\Omega(V)$ 的前提，兩者才都可簡寫 $O(VE)$。同步層數不變量較精確，卻必須支付維持兩層陣列的成本。若加「無改善提早停止」，空邊例只做一輪，必須另述這個版本。

最短路徑遞迴的歸納與負環判定

```
// Synchronous edge-count recurrence:
D[0,s]=0; D[0,v!=s]=infinity
D[k,v]=min(D[k-1,v], min_{(u,v) in E}(D[k-1,u]+w(u,v)))
// Allowed-intermediate recurrence, with k outermost:
F[k+1,i,j]=min(F[k,i,j],F[k,i,k]+F[k,k,j])
```

Bellman–Ford 的同步歸納：0邊僅能停在起點。至多k邊到v的走訪，若不到k邊已由舊值涵蓋，否則最後一邊是某u→v，其前綴至多k−1邊；枚舉所有最後一邊恰給遞迴式的兩個方向。有限最短值若存在，可移除非負環而不增長度，留下至多V−1邊的簡單路徑，所以V−1輪足夠。

額外輪仍有鬆弛，表示某條至多V邊走訪優於所有至多V−1邊走訪；該走訪必含重複頂點。若每個環都非負，移除環便得到不更長的短走訪而矛盾，故必有起點可達負環。反過來，若存在可達負環卻所有邊都滿足 d[v]≤d[u]+w(u,v)，沿負環把不等式加總會得0≤負數，矛盾。因此至少一邊仍能鬆弛。只有由這些受影響端點往外可達的頂點，其答案才是−∞；不相關分量不受影響。

最短路徑遞迴的歸納與負環判定 （續）

Floyd 的歸納依「允許的中介點集合」而非邊數。無相關負環時，可選不重複頂點的最短路徑。加入中介點k後，路徑要麼不用k，要麼拆成i→k與k→j，兩段只用舊集合；合併最優兩段又是合法走訪，故取最小給等號。原地更新可行，是本輪 D[k,k]=0 時 D[i,k]、D[k,j]不被降低；若已有負環，此有限距離論證不再適用，需改用可達性分類−∞。不能因同樣是三層迴圈就任意改成i最外層。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

最短路徑的條件比較

方法	必要條件	問題與成本
BFS	每邊相同正成本或只算邊數	單源 O(V+E)
DAG 拓樸鬆弛	有向無環，可含負邊	單源 O(V+E)
Dijkstra	非負邊權	單源，堆版本見〈Dijkstra 的優先佇列、舊項與前驅〉
Bellman–Ford	可含負邊	單源；原地 O(V+VE)，同步 O(V(V+E))；偵測可達負環
Floyd–Warshall	可含負邊	全點對 $\Theta(V^3)$

負權限制、圖是否無環、需要單源還是全點對，應先於資料結構偏好決定方法。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

瓶頸路徑的目標、鬆弛與完整追蹤

```
bottleneck(s):
    B[*]=infinity; parent[*]=none; B[s]=-infinity
    H.push((-infinity,s)); settled={}
    while H not empty:
        (bu,u)=H.pop_min()
        if u in settled or bu!=B[u]: continue
        settled.add(u)
        for (u,v,w) in adj[u]:
            candidate=max(B[u],w)
            if candidate<B[v]:
                B[v]=candidate; parent[v]=u; H.push((candidate,v))
```

瓶頸值定義為路徑最大邊權，目標是最小化這個最大值，即 $\min_{P:s\rightsquigarrow t}\max_{e\in P} w(e)$，不是把邊權相加。空路徑用−∞作max的單位元，不可達用+∞。對非負邊可把起點改0，但若容許負邊，起點0會把全負路徑的瓶頸誤抬高到0。

無向例s–p=6、p–t=6、s–q=2、q–t=9、p–q=7。取出s後B(p,q,t)=(6,2,∞)；取q後(6,2,9)、t.parent=q；取p後(6,2,6)、t.parent=p；最後t確定6，路徑s,p,t。最短總長卻選s,q,t得11，另一路總長12。相同資料的兩個目標產生不同答案，不能只換方法名稱。

瓶頸路徑的目標、鬆弛與完整追蹤 （續）

正確性仍用最小標籤確定：任一更佳路徑的第一個未確定點，其已確定前驅已給它不大於該路徑前綴最大值的候選；而延長路徑取max永不降低標籤，所以它的標籤不大於整條更佳路，與取出最小u矛盾。這個單調性對負邊也成立，與加法Dijkstra的非負前提不同。二元堆含舊項的界同前述Dijkstra；有indexed heap為O((V+E)log(V+1))。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

MST 路徑的瓶頸性質與門檻連通

```
threshold_bottleneck(s,t):
    require undirected graph and s!=t
    make_set(v) for all vertices
    for (u,v,w) in edges sorted by weight:
        union(u,v)
        if find(s)==find(t): return w
    return unreachable
```

無向圖取任一MST T，令其s到t唯一路徑上的最大邊權為β，選一條達β的邊e。移除e把T分成s、t兩側。若原圖有跨此割而更輕的邊f，用f換e會使生成樹總重下降，與T最小矛盾。因此任何s→t路徑跨割時至少用一條權重≥β的邊，瓶頸值不可能小於β；T的路徑本身達β，兩界相等。

這不表示MST路徑最小化總和。本章例子的MST為{s–q2,s–p6,p–t6}、總重14；s到t樹路徑總長12不是圖中最短11，但最大邊6確是最小瓶頸。MST也允許負邊，交換證明與max論證仍成立。此樹路徑性質不直接套到有向圖，因一般MST不是有向可達樹的定義。

MST 路徑的瓶頸性質與門檻連通 （續）

另一個等價觀點：答案是讓 s, t 首次在 $G_{\le \lambda}$ 連通的最小 λ。按權重由小到大union，第一次同集合時返回當前權重；對全部 V 個頂點 make_set 先花 Θ(V) 時間及 Θ(V) DSU 空間，故總時間 O(V+E log(E+1)+E α(V))。若另儲存待排序邊，邊緩衝為 Θ(E)，排序額外工作空間依具體排序法另計。E=0 而有 V 個孤立點時，演算法仍初始化全部頂點後回 unreachable，為 Θ(V)，不能把成本寫成零。這個方法只回答一對點而不計距離和；多次查詢可在MST上預處理路徑最大值，但還需另外設計資料結構與預處理成本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

多棵 MST 的權重層與必要邊

```
classify_MST_edges(G):
    require G undirected and connected
    DSU=singleton components
    for all equal-weight groups in increasing weight:
        Q=multigraph whose vertices are current DSU components
        for edge (u,v) in this group:
            if find(u)==find(v): mark edge as never_in_MST
            else: add its original edge ID between these two vertices in Q
        mark Q bridges as in_every_MST; other Q edges as optional
        for (u,v) in this group: union(u,v)
```

固定門檻 λ，令 c_λ 是只保留權重至多 λ 的邊之原圖分量數。任一 MST 中權重至多 λ 的邊必有恰 $n - c_\lambda$ 條：若在某個 $G_{\le\lambda}$ 分量內還不連通，沿該分量路徑找一條跨樹內分量且權重至多 λ 的邊，加進 MST 造成的環必含權重大於 λ 的邊；替換即更輕，矛盾。因這些邊本來無環，連通各分量後恰有 $n - c_\lambda$ 條。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

多棵 MST 的權重層與必要邊 (續)

相鄰權重層的計數相減，得每棵MST對每個權值選的邊數都相同；不代表選的是同一組邊，也不代表所有可替代邊都同權。先收縮嚴格較輕邊的分量，在當前等權multigraph Q中選各分量的生成森林。Q的bridge必選；在cycle上的邊可選也可不選；收縮後self-loop永不選。必須整組分類完才union，逐邊先union會把同權替代邊誤判成永不選。

Q可能有平行邊，雖原圖是簡單圖也一樣；兩個分量間的兩條平行邊彼此可替代。橋偵測需以edge ID略過父邊，而不是略過所有通向parent頂點的邊。排序加各Q的線性DFS，總$O(E \log(E+1))$，另加DSU攤銷成本。這把「找到一棵」提升成可證明某邊是否對所有最優解都必要。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

橋的 low-link 完整程序與證明

```
bridges(Q):
    tin[*]=unset; clock=0; result=[]
    for v in Q.vertices:
        if tin[v]==unset: dfs(v,none)
    return result
// dfs shares tin, low, clock, result with the enclosing routine.
dfs(v,parent_edge):
    tin[v]=low[v]=clock; clock+=1
    for (u,edge_id) in Q.adj[v]:
        if edge_id==parent_edge: continue
        if tin[u]==unset:
            dfs(u,edge_id); low[v]=min(low[v],low[u])
            if low[u]>tin[v]: result.append(edge_id)
        else: low[v]=min(low[v],tin[u])
```

tin記第一次發現時間；low[v]是由v子樹先走零或多條DFS樹邊向下，再至多走一條非父回邊可達的最小tin。這是無向圖的DFS量，不是最短路徑距離。若樹邊v→u有low[u]>tin[v]，u子樹沒有邊回到v或更上祖先，刪掉(v,u)即斷開；若low[u]≤tin[v]，存在繞過父邊的回連路徑，故不是橋。

橋的 low-link 完整程序與證明 （續）

數例：Q頂點X,Y,Z，X–Y有e1、e2兩條平行邊，Y–Z只有e3。從X以e1進Y，tin(X)=0、tin(Y)=1；在Y略過e1，但e2回X使low(Y)=0。進Z得tin(Z)=low(Z)=2，2>1所以e3是橋；返回X時low(Y)=0不大於0，e1不是橋，e2是回邊也不是橋。若把所有通向X的邊都略過，就錯把e1判成橋。

每個頂點只首次進入一次，每條無向邊的兩筆鄰接項各處理一次，時間Θ(|V_Q|+|E_Q|)、額外空間O(|V_Q|)。分類MST時只需建立各權重組有實際入射邊的商圖頂點，所有組的頂點數總和O(E)，不應每組配置整個原圖V大小陣列而仍宣稱總線性。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

AOV 網路與拓樸排序

- AOV 以頂點表示活動，有向邊表示先後依賴。合法排程要求圖為 DAG。
- Kahn 演算法把所有入度 0 的點入列。每次取一點輸出，移除其出邊，鄰點入度變 0 就入列。
- 不變量為剩餘入度只計尚未移除的邊。若還有點卻沒有入度 0，剩餘圖含有有向環。
- 輸出頂點數不足 V 即判定有環。時間 $\Theta(V+E)$。多個入度 0 點會產生多種合法次序。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

AOE 網路與最早事件時間

- AOE 以邊表示有工期的活動，以點表示事件。活動需在前一事件完成後才能開始。
- 本教材工期 $d(u,v)\ge 0$，圖為 DAG，採共同起點及終點。多起終點可用 0 工期虛擬邊連接。
- 沿拓樸序計算 $E[v]=\max_{(u,v)}(E[u]+d(u,v))$，起點 $E=0$。
- 這是 DAG 最長路徑，而非 Dijkstra 最短路徑。最大值來自所有前置事件都必須完成的限制。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

最晚事件、鬆弛與關鍵活動

- 終點 `L[t]=E[t]`，逆拓樸序計算 `L[u]=min_{(u,v)}(L[v]-d(u,v))`。
- 活動 `(u,v)` 的最早開始為 `E[u]`，最晚開始為 `L[v]-d(u,v)`，總鬆弛為兩者差。
- 零鬆弛活動為關鍵活動。可有多條關鍵路徑，不能只回報任意一條後說其餘活動都可延遲。
- 此模型假設資源不限且工期固定。共用人員、機台衝突或隨機工期需要額外排程模型。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

關鍵路徑數據

活動為 `s→a:3,s→b:2,a→t:4,b→t:6`。

- 最早事件為 `E[s]=0,E[a]=3,E[b]=2,E[t]=8`。
- 最晚事件為 `L[t]=8,L[a]=4,L[b]=2,L[s]=0`。
- `s→a` 與 `a→t` 的總鬆弛均為 1。`s→b` 與 `b→t` 的鬆弛為 0，關鍵路徑總工期 8。
- 兩個非關鍵活動各有 1 的總鬆弛，不代表可各自同時多耗 1 且仍不影響工期。共享同一路徑的鬆弛不能重複花用。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

拓樸排序與 AOE 的完整正反向計算

```
Kahn():
    indegree=counts of all incoming edges; Q=all zero-indegree vertices
    order=[]
    while Q not empty:
        u=Q.pop(); order.append(u)
        for (u,v) in outgoing: indegree[v]-=1; if indegree[v]==0: Q.push(v)
    require len(order)==V
AOE(order):
    E[*]=-infinity; E[start]=0
    for u in order:
        for (u,v,d): E[v]=max(E[v],E[u]+d)
    L[*]=infinity; L[finish]=E[finish]
    for u in reverse(order):
        for (u,v,d): L[u]=min(L[u],L[v]-d)
```

AOE同一s→a3,s→b2,a→t4,b→t6：初入度(s,a,b,t)=(0,1,1,2)，輸出s後(0,0,0,2)，輸出a後t降1，輸出b後t降0，最後t，得s,a,b,t。正向E初(0,−∞,−∞,−∞)，處理s得(0,3,2,−∞)，a後t=7，b後t=8。逆向L初t=8，處理b得L[b]=2，處理a得4，處理s得min(4−3,2−2)=0。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

拓樸排序與 AOE 的完整正反向計算 （續）

四活動(最早開始,最晚開始,總鬆弛)為s→a:(0,1,1)，s→b:(0,0,0)，a→t:(3,4,1)，b→t:(2,2,0)。若活動工期固定且資源不限，零鬆弛路s,b,t決定總工期8；a路7共享一單位餘裕，兩活動各加1後變9，不能重複消耗相同餘裕。無共同start/finish時先加零工期虛邊，並確認每項欲分析活動位於起終可達範圍。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

圖像、驗證與來源

- 本章練習與解答分檔，要求追蹤狀態並交代成立條件。程式實驗對照 Dijkstra、Bellman–Ford 與 Floyd–Warshall 的結果；Floyd 範例只接受非負權圖，作交叉驗證用，不宣稱實作完整負環分類。
- 搜尋與單源最短路徑條件核對 MIT 6.006 2020 L9–L14。MST 可對照 MIT 6.046J 2015 課程講義。
- AOV/AOE 命名依使用者指定，公式、數據及資源限制分析由本教材獨立整理。未查閱原書圖例或解答。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Ellis Horowitz, Sartaj Sahni, Dinesh Mehta. Fundamentals of Data Structures in C++, 2nd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study